

ABSTRACT

The Optimal Binary Search Tree (OBST) is an advanced data structure technique used to minimize the average search cost by organizing keys based on their access frequencies or probabilities. This project focuses on the design and development of an interactive application that constructs and visualizes an Optimal Binary Search Tree using the Dynamic Programming approach.

The system is developed using Python with the Tkinter library to provide a user-friendly graphical interface. Users can input keys along with their corresponding frequencies or probabilities, and the application computes the optimal tree structure that ensures efficient searching. The project also generates a standard balanced Binary Search Tree for comparison, highlighting the performance improvement achieved through optimization.

The application visually represents both trees, displays the dynamic programming matrix used in computation, and provides a step-by-step trace of the algorithm. These features help users understand the internal working of the OBST algorithm in a clear and interactive manner.

The results demonstrate that the Optimal Binary Search Tree significantly reduces the average search cost compared to a standard Binary Search Tree, especially when access frequencies vary. This project serves as an effective educational tool for understanding dynamic programming and optimized data structures, while also showcasing practical implementation and visualization techniques.

LIST OF ABBREVIATIONS

OBST	Optimal Binary Search Tree
BST	Binary Search Tree
DP	Dynamic Programming
GUI	Graphical User Interface
UI	User Interface
CPU	Central Processing Unit
RAM	Random Access Memory
OS	Operating System
IDE	Integrated Development Environment
Tkinter	Toolkit Interface (Python GUI Library)
$e[i][j]$	Cost Table (Expected Search Cost)
$w[i][j]$	Weight Table (Sum of Frequencies)
$root[i][j]$	Root Table (Optimal Root Index)

CHAPTER 1

INTRODUCTION

1.1 Overview

The Optimal Binary Search Tree (OBST) is an important concept in **data structures and algorithms**, used to improve the efficiency of searching operations. In a normal Binary Search Tree (BST), the structure depends only on the order of insertion, which may not always lead to optimal performance. However, OBST takes into account the **frequency or probability of searching each key**, and constructs a tree that minimizes the overall search cost.

This project presents an **interactive visualization tool** for constructing an Optimal Binary Search Tree using **Dynamic Programming**. The application is developed using Python's Tkinter library and provides a user-friendly interface to input keys and their frequencies, compute the optimal tree, and visualize the results. It also compares the OBST with a standard balanced BST to highlight performance differences.

1.2 Problem Statement

In many real-world applications such as databases, dictionaries, and search engines, some elements are accessed more frequently than others. A standard Binary Search Tree does not consider this frequency, which can lead to inefficient search operations.

The problem is to design a Binary Search Tree such that the **average search cost is minimized**, taking into account the frequency of each key. This requires determining the optimal arrangement of nodes so that frequently accessed keys are placed closer to the root.

1.3 Aim of the Project

The aim of this project is to design and develop an interactive application that demonstrates the construction of an **Optimal Binary Search Tree (OBST)** using the Dynamic Programming approach. The project focuses on minimizing the average search cost by organizing keys based on their frequencies or probabilities.

It also aims to provide a clear visual understanding of how OBST differs from a standard Binary Search Tree by displaying both structures and comparing their performance. Additionally, the project intends to help users learn the step-by-step execution of the algorithm through dynamic programming tables and visual traces, making the concept easier to understand and analyze.

1.4 Scope of the Project

This project is designed as an educational and analytical tool for understanding the concept of Optimal Binary Search Trees. It is useful for students, learners, and educators in the field of computer science.

The scope of the project includes:

- Construction and visualization of OBST for a given set of keys
- Comparison between OBST and balanced BST based on search cost
- Step-by-step explanation of the Dynamic Programming approach
- Interactive features such as subtree highlighting and algorithm tracing

However, the project is limited to:

- Static input (manual entry of keys and frequencies)
- In-memory computation (no database integration)
- Desktop-based execution using Tkinter

CHAPTER 2

SOFTWARE AND HARDWARE SPECIFICATIONS

2.1 Hardware Requirements

The hardware requirements for this project are minimal, as it is a lightweight desktop application. The following configuration is sufficient for smooth execution:

- **Processor:** Intel Core i3 or above
- **RAM:** Minimum 4 GB
- **Hard Disk:** At least 500 MB of free space
- **Display:** 1366×768 resolution or higher
- **Input Devices:** Keyboard and Mouse

2.2 Software Requirements

The software requirements for developing and running this project are as follows:

- **Operating System:** Windows 10 / Windows 11 / Linux / macOS
- **Programming Language:** Python 3.x
- **Libraries Used:**
 - Tkinter (for GUI development)
 - Math (for calculations)
 - Time (for animation and delays)
- **IDE/Editor:** Visual Studio Code / PyCharm / IDLE
- **Python Version:** Python 3.7 or above

CHAPTER 3

PROJECT DESCRIPTION

3.1 Introduction

In today's digital world, efficient data retrieval plays a crucial role in the performance of computer systems. Data structures are designed not only to store data but also to ensure that operations such as searching, insertion, and deletion are performed efficiently. Among these operations, searching is one of the most frequently performed tasks in applications such as databases, file systems, search engines, and information retrieval systems.

A Binary Search Tree (BST) is a widely used data structure that allows efficient searching, insertion, and deletion operations with an average time complexity of $O(\log n)$. However, the efficiency of a BST largely depends on its structure. In many cases, the structure of a BST is determined by the order in which elements are inserted, which may lead to an unbalanced tree. In the worst case, the BST can become skewed, resulting in a time complexity of $O(n)$, which significantly degrades performance.

To overcome these limitations, the concept of an Optimal Binary Search Tree (OBST) is introduced. Unlike a standard BST, an OBST is constructed by considering the frequency or probability of access of each key. The main objective of an OBST is to minimize the expected or average search cost by placing frequently accessed keys closer to the root of the tree. This ensures that commonly searched elements can be found quickly, improving the overall efficiency of the system.

The construction of an Optimal Binary Search Tree is a classic problem that can be effectively solved using the Dynamic Programming approach. Dynamic Programming is a powerful technique used to solve complex problems by breaking them down into smaller subproblems and storing their results to avoid redundant computations. In the case of OBST, the algorithm computes the optimal cost for all possible subtrees and selects the best root for each subtree in a bottom-up manner. This results in a globally optimal solution.

The OBST problem involves calculating the expected search cost based on the depth of each node and its associated frequency. The deeper a node is in the tree, the higher the cost of accessing it. Therefore, the goal is to arrange the nodes in such a way that frequently

accessed nodes have smaller depths, while less frequently accessed nodes are placed deeper in the tree. This trade-off leads to an optimized structure that minimizes the total weighted search cost.

Despite its importance, the OBST concept can be difficult for students to understand due to its reliance on dynamic programming and matrix-based computations. The process involves constructing cost tables, weight tables, and root tables, which may appear abstract without proper visualization. To address this challenge, this project focuses on developing an interactive visualization tool that helps users understand the working of the OBST algorithm in a clear and intuitive manner.

This project implements the OBST algorithm using Python and provides a graphical user interface (GUI) using the Tkinter library. The application allows users to input keys along with their frequencies or probabilities and then computes the optimal binary search tree. The system not only generates the OBST but also constructs a standard balanced BST for comparison purposes. This comparison helps users observe the difference in performance between an optimized tree and a regular tree.

One of the key features of this project is its ability to visually represent both the OBST and the standard BST. The trees are displayed graphically, showing the hierarchical structure of nodes and their relationships. Additionally, the project includes a dynamic programming matrix visualization, where each cell represents the cost and root of a particular subtree. This matrix is displayed using a heatmap, making it easier to identify patterns and understand how the algorithm progresses.

Another important feature of the project is the step-by-step algorithm trace. This feature allows users to observe how the OBST is constructed by examining each subproblem and the corresponding decisions made by the algorithm. Users can navigate through each step, view intermediate calculations, and understand how the optimal solution is derived. This makes the learning process more interactive and engaging.

The project is designed with a modular architecture, separating input handling, algorithm computation, tree construction, and visualization. This modular approach improves the readability, maintainability, and scalability of the system. The use of Python ensures simplicity and flexibility, while Tkinter provides an effective platform for developing desktop-based graphical applications.

In summary, this project serves as both a functional tool and an educational aid for understanding the Optimal Binary Search Tree. By combining algorithm implementation with visualization, it bridges the gap between theoretical concepts and practical understanding. The project highlights the importance of optimization in data structures and demonstrates how dynamic programming can be applied to solve real-world problems efficiently.

3.2 Architecture Diagram

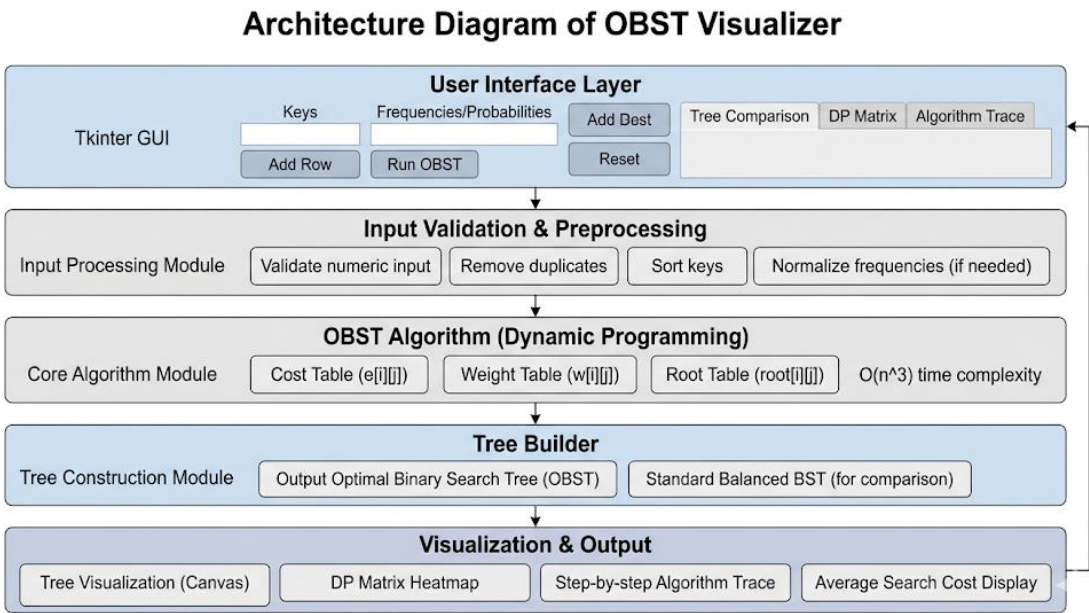


Figure 3.1: Architecture Diagram

The architecture of the Optimal Binary Search Tree (OBST) Visualizer is designed in a modular and layered manner to ensure clarity, efficiency, and ease of understanding. The system is divided into multiple components, each responsible for a specific function, and all components work together to process input, perform computation, and display results.

At the top level, the **User Interface (UI)** is developed using the Tkinter library. It acts as the interaction layer between the user and the system. The user provides input in the form of keys and their corresponding frequencies or probabilities. The interface also includes control buttons such as Run, Add Row, and Reset, along with tabs for viewing tree comparison, dynamic programming matrix, and algorithm trace.

The next layer is the **Input Processing Module**, which is responsible for validating and organizing the user input. It ensures that the entered values are numeric, removes duplicate keys, and sorts the keys in ascending order. This preprocessing step is important to maintain the correctness of the Binary Search Tree properties and to prepare the data for the algorithm.

The core component of the system is the **OBST Algorithm Module**, which implements the Dynamic Programming approach. This module computes three main tables: the cost table, weight table, and root table. These tables store intermediate results and help in determining the optimal structure of the tree. By evaluating all possible subtrees and selecting the minimum cost configuration, this module ensures that the final tree is optimal.

Following this, the **Tree Construction Module** uses the root table generated by the algorithm to build the Optimal Binary Search Tree. In addition to the OBST, the system also constructs a standard balanced Binary Search Tree for comparison purposes. This helps in analyzing the effectiveness of the OBST.

Finally, the **Visualization Module** presents the output in a graphical format. It displays the constructed trees, the dynamic programming matrix with cost values, and a step-by-step trace of the algorithm. It also shows the average search cost, allowing users to compare performance.

Overall, the architecture ensures a smooth flow of data from input to output, making the system efficient, interactive, and easy to understand.

3.3 Algorithm Used

Optimal Binary Search Tree (OBST) using Dynamic Programming

The Optimal Binary Search Tree (OBST) algorithm is used to construct a Binary Search Tree such that the average search cost is minimized. Unlike a standard Binary Search Tree, which depends only on the order of insertion, OBST takes into account the frequency or probability of accessing each key.

The main idea is to arrange the keys in such a way that frequently accessed keys are placed closer to the root, thereby reducing the number of comparisons required during search operations.

Concept of OBST

In a Binary Search Tree, the cost of searching a key depends on its depth. A node closer to the root requires fewer comparisons, while deeper nodes require more comparisons. Therefore, to minimize the total search cost, frequently accessed keys should be positioned at lower depths.

The OBST algorithm calculates the minimum weighted search cost by considering all possible Binary Search Trees and selecting the one with the least cost.

Dynamic Programming Approach

The OBST problem is solved using Dynamic Programming because:

- It has overlapping subproblems
- It follows optimal substructure
- It avoids redundant calculations by storing intermediate results

Three tables are used in the algorithm:

1. Cost Table ($e[i][j]$)
Stores the minimum search cost for keys from index i to j .
2. Weight Table ($w[i][j]$)
Stores the sum of frequencies from index i to j .
3. Root Table ($root[i][j]$)
Stores the root that gives minimum cost for the subtree.

Algorithm Steps

1. Initialization
 - For all i , set:
 - $e[i][i-1] = 0, e[i][i] = 0, e[i][i+1] = 0$

- $w[i][i-1] = 0$

2. Compute for Subtrees of Increasing Length

- For length $L = 1$ to n :

- For each range $[i, j]$:

- $j = i + L - 1$

3. Calculate Weight

$$w[i][j] = w[i][j-1] + \text{freq}[j]$$

4. Find Optimal Root

- For each possible root r from i to j :

$$\text{cost} = e[i][r-1] + e[r+1][j] + w[i][j]$$

$$w[i][j] \text{cost} = e[i][r-1] + e[r+1][j] + w[i][j]$$

- Select the root that gives the minimum cost
- Store it in $\text{root}[i][j]$

5. Construct the Tree

- Use the root table recursively to build the final OBST

Pseudocode

for $i = 1$ to $n+1$:

$$e[i][i-1] = 0$$

$$w[i][i-1] = 0$$

for $L = 1$ to n :

for $i = 1$ to $n-L+1$:

$$j = i + L - 1$$

$$e[i][j] = \infty$$

$$w[i][j] = w[i][j-1] + \text{freq}[j]$$

for $r = i$ to j :

$$t = e[i][r-1] + e[r+1][j] + w[i][j]$$

if $t < e[i][j]$:

$$e[i][j] = t$$

$$root[i][j] = r$$

Time and Space Complexity

- Time Complexity: $O(n^3)$
- Space Complexity: $O(n^2)$

Working in the Project

In this project:

- The algorithm takes user input (keys and frequencies)
- Computes DP tables (e, w, root)
- Constructs the Optimal BST
- Displays:
 - Tree structure
 - DP matrix
 - Step-by-step execution

3.4 Advantages of Algorithm Used

The Optimal Binary Search Tree (OBST) algorithm offers several advantages over traditional Binary Search Tree methods, especially in applications where search operations are frequent.

One of the major advantages of the OBST algorithm is that it **minimizes the average search cost**. By considering the frequency or probability of accessing each key, the algorithm ensures that frequently accessed elements are placed closer to the root of the tree. This significantly reduces the number of comparisons required during search operations.

Another important advantage is its use of **Dynamic Programming**, which avoids redundant calculations by storing intermediate results. This improves efficiency when compared to brute-force methods that evaluate all possible tree structures. The algorithm ensures an optimal solution by solving smaller subproblems and combining their results.

The OBST algorithm also provides a **globally optimal tree structure**, unlike standard Binary Search Trees where the structure depends on the order of insertion. This makes OBST more reliable and efficient for applications where data access patterns are known in advance.

Additionally, the algorithm is highly useful in real-world applications such as **database indexing, dictionary implementation, and symbol tables**, where certain elements are accessed more frequently than others. By optimizing the structure based on access frequency, system performance can be improved.

Another advantage is that the OBST algorithm ensures **better performance consistency**. Even if the input data varies, the algorithm always produces the most efficient tree structure for the given frequencies.

Finally, the OBST approach helps in **better understanding of optimization techniques** in computer science. It demonstrates how dynamic programming can be applied to solve complex problems efficiently and is therefore valuable from both practical and educational perspectives.

3.5 Explanation of the Project

The project is developed using Python and Tkinter to provide an interactive environment for understanding OBST.

Working Process:

1. User Input
 - User enters keys and frequencies
 - Data is validated and sorted
2. Processing
 - OBST algorithm computes:
 - Cost matrix
 - Root table
3. Tree Construction

- OBST is built using root table
- Balanced BST is also generated for comparison

4. Visualization

- Trees are drawn using canvas
- DP matrix is displayed with color coding
- Clicking cells highlights subtrees

5. Algorithm Trace

- Step-by-step explanation of DP computation
- Shows how optimal root is selected

3.6 Output Results

The system produces clear and interactive outputs that help users understand the working and efficiency of the Optimal Binary Search Tree (OBST). The main output includes the **graphical representation of the OBST**, where keys are arranged based on their frequencies, ensuring that frequently accessed elements are placed closer to the root.

For comparison, the system also generates a **standard balanced Binary Search Tree**, which does not consider frequencies. By displaying both trees side by side, users can easily observe the structural differences and understand how OBST improves search efficiency.

The application also calculates and displays the **average search cost** for both trees. Typically, the OBST has a lower cost, demonstrating its effectiveness in minimizing search operations.

Additionally, the system shows the **Dynamic Programming matrix**, which contains the computed costs and optimal roots for different subtrees. A step-by-step trace is also provided to explain how the algorithm arrives at the final solution.

Overall, the outputs clearly illustrate that the OBST provides a more efficient structure compared to a standard BST, especially when access frequencies vary.

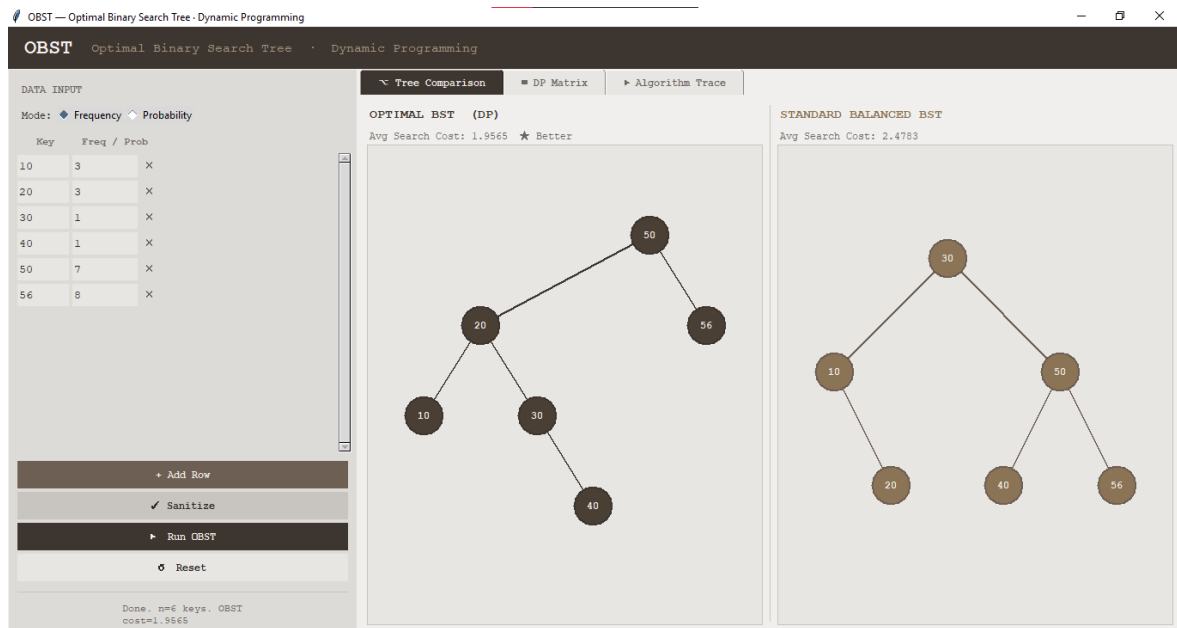


Figure 3.2: OBST and Balanced BST Output Comparison

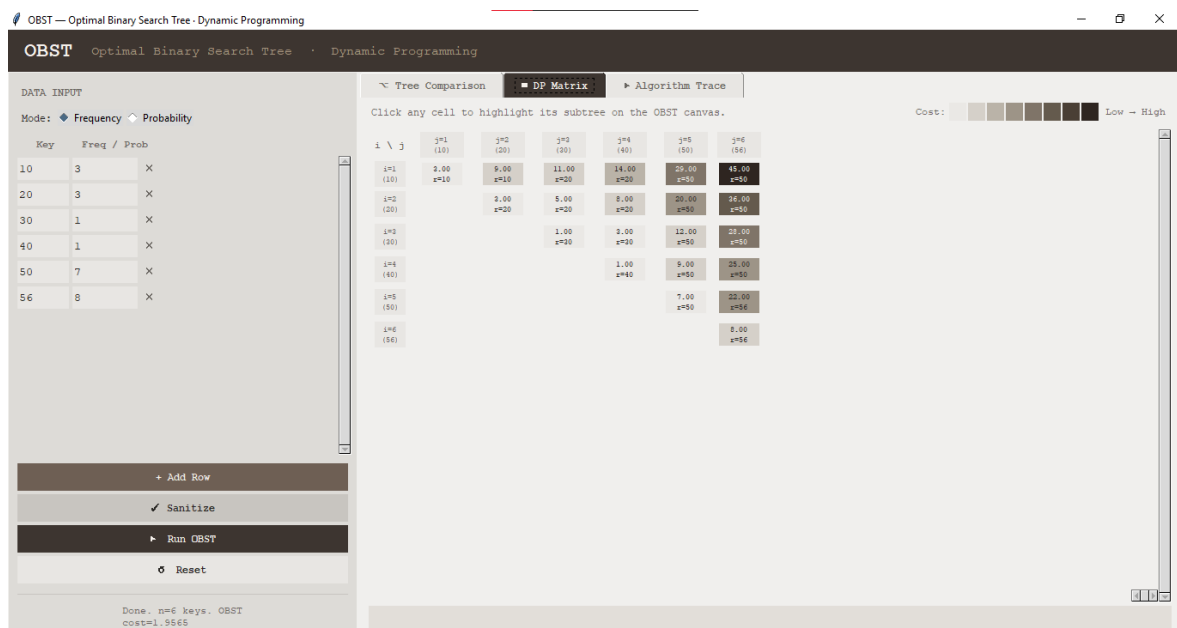


Figure 3.3: Dynamic Programming Matrix Output

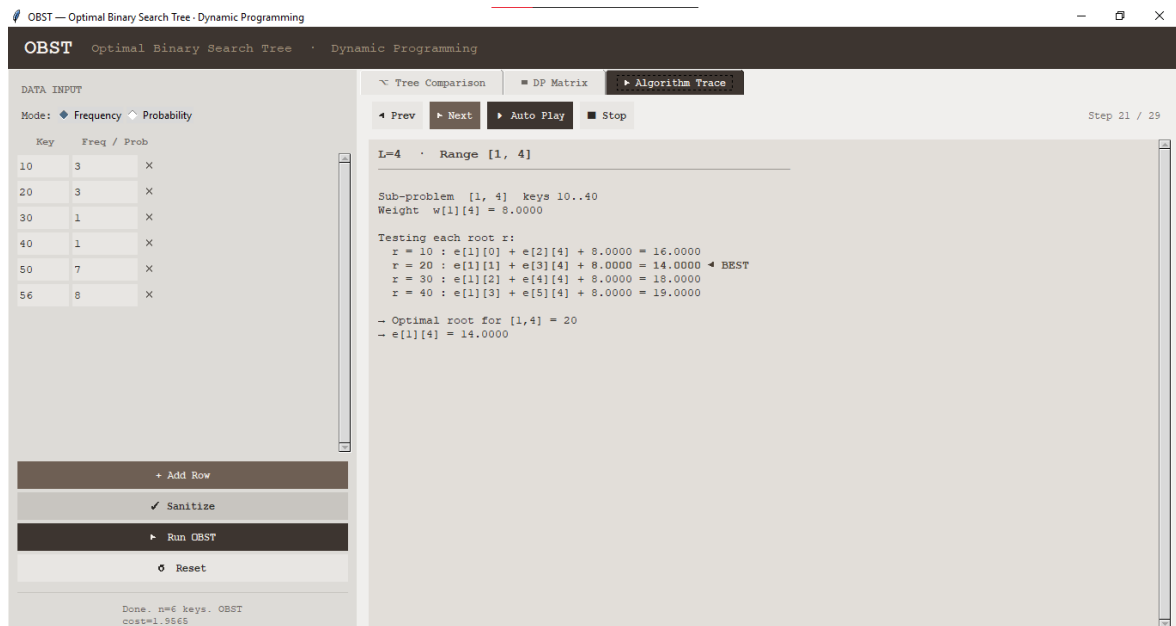


Figure 3.4: Algorithm Trace Visualization

3.7 Conclusion

The project successfully demonstrates the construction and visualization of the Optimal Binary Search Tree (OBST) using the Dynamic Programming approach. It clearly shows how considering the frequency or probability of keys helps in minimizing the average search cost and improving overall efficiency.

Through graphical representation and comparison with a standard Binary Search Tree, the project highlights the advantages of OBST in organizing data more effectively. The inclusion of the dynamic programming matrix and step-by-step trace further enhances understanding of the algorithm.

Overall, the project serves as a useful educational tool that bridges the gap between theoretical concepts and practical implementation. It emphasizes the importance of optimization techniques in data structures and provides a clear insight into efficient search methods.

APPENDIX

Source Code

```
import tkinter as tk

from tkinter import ttk, messagebox, font as tkfont

import math

import time

BG      = "#F0EFED"    # warm off-white background

PANEL   = "#E8E6E3"    # slightly darker panel fill

SIDEBAR = "#DDDBD7"    # sidebar

BORDER  = "#C8C5C0"    # borders / separators

TEXT_DARK = "#2B2926"  # primary text

TEXT_MID  = "#6B6762"  # secondary text

TEXT_LIGHT = "#9A9691" # muted text

ACC_DARK  = "#3D3530"  # very dark brown — edges & headers

ACC_MED   = "#6E5F54"  # mid-brown — selected nodes

ACC_LIGHT = "#A8957F"  # tan — hover / highlight

OBST_NODE = "#4A3F35"  # OBST node fill

OBST_TEXT = "#F5F0EB"  # OBST node label

BST_NODE  = "#8B7355"

BST_TEXT  = "#F5F0EB"

HEAT =
["#EAE8E5", "#D5D0C9", "#BAB3A8", "#9D9487", "#7F7468", "#645A4D", "#4A3F35",
"#2E2520"]

TRACE_BG = "#E2DED9"
```

```

TRACE_HL = "#6E5F54"

def obst_dp(keys, freq):

    n = len(keys)

    e = [[0.0]*(n+2) for _ in range(n+2)]

    w = [[0.0]*(n+2) for _ in range(n+2)]

    root = [[0]*(n+1) for _ in range(n+1)]

    for i in range(1, n+2):

        e[i][i-1] = 0.0

        w[i][i-1] = 0.0

    for length in range(1, n+1):      # chain length

        for i in range(1, n-length+2): # start index

            j = i + length - 1        # end index

            e[i][j] = float('inf')

            w[i][j] = w[i][j-1] + freq[j-1]

            for r in range(i, j+1):

                t = e[i][r-1] + e[r+1][j] + w[i][j]

                if t < e[i][j]:

                    e[i][j] = t

                    root[i][j] = r

    return e, w, root

def build_tree(root_table, i, j, keys):

    if i > j:

        return None

    r = root_table[i][j]

```

```

return {

    "key": keys[r-1],

    "r_idx": r,

    "left": build_tree(root_table, i, r-1, keys),

    "right": build_tree(root_table, r+1, j, keys),

}

def build_balanced_bst(keys):

    def helper(lo, hi):

        if lo > hi:

            return None

        mid = (lo + hi) // 2

        return {

            "key": keys[mid],

            "r_idx": mid+1,

            "left": helper(lo, mid-1),

            "right": helper(mid+1, hi),

        }

    return helper(0, len(keys)-1)

def tree_height(node):

    if node is None:

        return 0

    return 1 + max(tree_height(node["left"]), tree_height(node["right"]))

def assign_positions(node, depth=0, counter=[0]):

    if node is None:

```

```

        return

    assign_positions(node["left"], depth+1, counter)

    node["x_order"] = counter[0]

    node["depth"] = depth

    counter[0] += 1

    assign_positions(node["right"], depth+1, counter)

def avg_search_cost(node, keys, freq, depth=1):

    if node is None:

        return 0.0

    total_freq = sum(freq)

    p = freq[node["r_idx"]-1] / total_freq if total_freq else 0

    return p * depth + avg_search_cost(node["left"], keys, freq, depth+1) \

        + avg_search_cost(node["right"], keys, freq, depth+1)

class OBSTApp(tk.Tk):

    def __init__(self):

        super().__init__()

        self.title("OBST — Optimal Binary Search Tree · Dynamic Programming")

        self.configure(bg=BG)

        self.geometry("1400x860")

        self.minsize(1100, 720)

        self._keys_entries = []

        self._freq_entries = []

        self._row_frames = []

        self._dp_cells = {}

```

```

self._trace_steps = []

self._trace_idx = 0

self._trace_running = False

self._last_keys = []

self._last_freq = []

self._last_root = None

self._build_ui()

self._add_row()

self._add_row()

self._add_row()

def _build_ui(self):

    bar = tk.Frame(self, bg=ACC_DARK, height=46)

    bar.pack(fill="x", side="top")

    tk.Label(bar, text="OBST", bg=ACC_DARK, fg="#F5F0EB",

             font=("Courier", 18, "bold")).pack(side="left", padx=16, pady=8)

    tk.Label(bar, text="Optimal Binary Search Tree · Dynamic Programming",

             bg=ACC_DARK, fg=ACC_LIGHT,

             font=("Courier", 11)).pack(side="left", pady=8)

    paned = tk.PanedWindow(self, orient="horizontal",

                           bg=BG, sashwidth=4, sashrelief="flat")

    paned.pack(fill="both", expand=True, padx=0, pady=0)

    self._left = tk.Frame(paned, bg=SIDEBAR, width=260)

    paned.add(self._left, minsize=230)

    self._center = tk.Frame(paned, bg=BG)

```

```

paned.add(self._center, minsize=500)

self._build_left_panel()

self._build_center_notebook()

def _build_left_panel(self):

    p = self._left

    tk.Label(p, text="DATA INPUT", bg=SIDEBAR, fg=TEXT_MID,

             font=("Courier", 9, "bold")).pack(anchor="w", padx=12, pady=(12,4))

    mf = tk.Frame(p, bg=SIDEBAR)

    mf.pack(fill="x", padx=12, pady=4)

    tk.Label(mf, text="Mode:", bg=SIDEBAR, fg=TEXT_DARK,

             font=("Courier", 9)).pack(side="left")

    self._mode = tk.StringVar(value="freq")

    ttk.Radiobutton(mf, text="Frequency", variable=self._mode,

                   value="freq").pack(side="left", padx=4)

    ttk.Radiobutton(mf, text="Probability", variable=self._mode,

                   value="prob").pack(side="left")

    hf = tk.Frame(p, bg=SIDEBAR)

    hf.pack(fill="x", padx=12, pady=(6,2))

    tk.Label(hf, text="Key", bg=SIDEBAR, fg=TEXT_MID,

             font=("Courier", 8, "bold"), width=8).pack(side="left")

    tk.Label(hf, text="Freq / Prob", bg=SIDEBAR, fg=TEXT_MID,

             font=("Courier", 8, "bold")).pack(side="left", padx=8)

    outer = tk.Frame(p, bg=SIDEBAR)

    outer.pack(fill="both", expand=True, padx=6, pady=2)

```

```

canvas = tk.Canvas(outer, bg=SIDEBAR, highlightthickness=0)

sb = ttk.Scrollbar(outer, orient="vertical", command=canvas.yview)

self._rows_frame = tk.Frame(canvas, bg=SIDEBAR)

self._rows_frame.bind("<Configure>",

    lambda e: canvas.configure(scrollregion=canvas.bbox("all")))

canvas.create_window((0,0), window=self._rows_frame, anchor="nw")

canvas.configure(yscrollcommand=sb.set)

canvas.pack(side="left", fill="both", expand=True)

sb.pack(side="right", fill="y")

bf = tk.Frame(p, bg=SIDEBAR)

bf.pack(fill="x", padx=10, pady=6)

self._btn(bf, "+ Add Row", self._add_row, ACC_MED,
"#F5F0EB").pack(fill="x", pady=2)

self._btn(bf, "✓ Sanitize", self._sanitize, BORDER,
TEXT_DARK).pack(fill="x", pady=2)

sep = tk.Frame(p, bg=BORDER, height=1)

sep.pack(fill="x", padx=10, pady=4)

self._btn(bf, "▶ Run OBST", self._run_obst, ACC_DARK,
"#F5F0EB").pack(fill="x", pady=2)

self._btn(bf, "↺ Reset", self._reset_all, PANEL,
TEXT_DARK).pack(fill="x", pady=2)

self._status = tk.Label(p, text="", bg=SIDEBAR, fg=TEXT_MID,

    font=("Courier", 8), wraplength=220, justify="left")

self._status.pack(fill="x", padx=12, pady=4)

def _btn(self, parent, text, cmd, bg, fg):

```

```

return tk.Button(parent, text=text, command=cmd,

                 bg=bg, fg=fg, relief="flat",

                 font=("Courier", 9, "bold"),

                 cursor="hand2", padx=6, pady=5,

                 activebackground=ACC_LIGHT, activeforeground=TEXT_DARK,

                 bd=0)

def _add_row(self, key_val="", freq_val=""):

    row = len(self._keys_entries) + 1

    f = tk.Frame(self._rows_frame, bg=SIDEBAR)

    f.pack(fill="x", pady=2, padx=4)

    ke = tk.Entry(f, width=7, bg=PANEL, fg=TEXT_DARK, relief="flat",

                 font=("Courier", 10), insertbackground=TEXT_DARK)

    ke.pack(side="left", padx=(0,4), ipady=3)

    ke.insert(0, key_val)

    fe = tk.Entry(f, width=9, bg=PANEL, fg=TEXT_DARK, relief="flat",

                 font=("Courier", 10), insertbackground=TEXT_DARK)

    fe.pack(side="left", padx=(0,4), ipady=3)

    fe.insert(0, freq_val)

    del_btn = tk.Button(f, text="X", bg=SIDEBAR, fg=TEXT_MID,

                       relief="flat", font=("Courier", 8), cursor="hand2",

                       command=lambda ff=f, ke=ke, fe=fe: self._del_row(ff, ke, fe),

                       bd=0, padx=2)

    del_btn.pack(side="left")

```

```

self._keys_entries.append(ke)

self._freq_entries.append(fe)

self._row_frames.append(f)

def _del_row(self, frame, ke, fe):

    idx = self._keys_entries.index(ke)

    self._keys_entries.pop(idx)

    self._freq_entries.pop(idx)

    self._row_frames.pop(idx)

    frame.destroy()

def _sanitize(self):

    raw = []

    for ke, fe in zip(self._keys_entries, self._freq_entries):

        k = ke.get().strip()

        f = fe.get().strip()

        if k and f:

            try:

                raw.append((float(k), float(f)))

            except ValueError:

                messagebox.showerror("Validation Error",

                                    f'Non-numeric value: key='{k}' freq='{f}''

                return

    if not raw:

        messagebox.showwarning("Empty", "Add at least one key/frequency pair.")

    return

```

```

seen = set()

unique = []

for k, f in sorted(raw, key=lambda x: x[0]):

    if k not in seen:

        seen.add(k)

        unique.append((k, f))

for i, frame in enumerate(self._row_frames):

    frame.destroy()

self._keys_entries.clear()

self._freq_entries.clear()

self._row_frames.clear()

for k, f in unique:

    kstr = str(int(k)) if k == int(k) else str(k)

    fstr = str(int(f)) if f == int(f) else str(round(f, 4))

    self._add_row(kstr, fstr)

self._status.config(text=f"Sanitized: {len(unique)} unique sorted keys.")

def _build_center_notebook(self):

    style = ttk.Style()

    style.theme_use("default")

    style.configure("TNotebook",      background=BG, borderwidth=0)

    style.configure("TNotebook.Tab",   background=PANEL,
foreground=TEXT_MID,

                                font=("Courier", 9, "bold"), padding=[12, 5])

    style.map("TNotebook.Tab",

```

```

        background=[("selected", ACC_DARK)],

        foreground=[("selected", "#F5F0EB"))

self._nb = ttk.Notebook(self._center)

self._nb.pack(fill="both", expand=True, padx=0, pady=0)

self._tab_tree = tk.Frame(self._nb, bg=BG)

self._nb.add(self._tab_tree, text=" ≡ Tree Comparison ")

self._tab_dp = tk.Frame(self._nb, bg=BG)

self._nb.add(self._tab_dp, text=" ▣ DP Matrix ")

self._tab_trace = tk.Frame(self._nb, bg=BG)

self._nb.add(self._tab_trace, text=" ▷ Algorithm Trace ")

self._build_tree_tab()

self._build_dp_tab()

self._build_trace_tab()

def _build_tree_tab(self):

    t = self._tab_tree

    cf = tk.Frame(t, bg=BG)

    cf.pack(fill="both", expand=True)

    lf = tk.Frame(cf, bg=BG)

    lf.pack(side="left", fill="both", expand=True, padx=8, pady=8)

    tk.Label(lf, text="OPTIMAL BST (DP)", bg=BG, fg=ACC_DARK,

        font=("Courier", 11, "bold")).pack(anchor="w", pady=(4,2))

    self._obst_cost_lbl = tk.Label(lf, text="Avg Search Cost: —",

        bg=BG, fg=TEXT_MID, font=("Courier", 9))

    self._obst_cost_lbl.pack(anchor="w")

```

```

self._canvas_obst = tk.Canvas(lf, bg=PANEL, highlightthickness=1,
                               highlightbackground=BORDER)

self._canvas_obst.pack(fill="both", expand=True)

sep = tk.Frame(cf, bg=BORDER, width=1)

sep.pack(side="left", fill="y", pady=12)

rf = tk.Frame(cf, bg=BG)

rf.pack(side="left", fill="both", expand=True, padx=8, pady=8)

tk.Label(rf, text="STANDARD BALANCED BST", bg=BG, fg=BST_NODE,
         font=("Courier", 11, "bold")).pack(anchor="w", pady=(4,2))

self._bst_cost_lbl = tk.Label(rf, text="Avg Search Cost: —",
                              bg=BG, fg=TEXT_MID, font=("Courier", 9))

self._bst_cost_lbl.pack(anchor="w")

self._canvas_bst = tk.Canvas(rf, bg=PANEL, highlightthickness=1,
                              highlightbackground=BORDER)

self._canvas_bst.pack(fill="both", expand=True)

def _build_dp_tab(self):

    t = self._tab_dp

    # info label at top

    info = tk.Frame(t, bg=BG)

    info.pack(fill="x", padx=10, pady=6)

    tk.Label(info, text="Click any cell to highlight its subtree on the OBST canvas.",
             bg=BG, fg=TEXT_MID, font=("Courier", 9)).pack(side="left")

    lg = tk.Frame(info, bg=BG)

    lg.pack(side="right")

```

```

tk.Label(lg, text="Cost:", bg=BG, fg=TEXT_MID, font=("Courier",
8)).pack(side="left")

for i, h in enumerate(HEAT):

    tk.Label(lg, bg=h, width=2, relief="flat").pack(side="left", padx=1)

    tk.Label(lg, text="Low → High", bg=BG, fg=TEXT_MID, font=("Courier",
8)).pack(side="left", padx=4)

    outer = tk.Frame(t, bg=BG)

    outer.pack(fill="both", expand=True, padx=10, pady=4)

    self._dp_canvas = tk.Canvas(outer, bg=BG, highlightthickness=0)

    hscroll = ttk.Scrollbar(outer, orient="horizontal",

        command=self._dp_canvas.xview)

    vscroll = ttk.Scrollbar(outer, orient="vertical",

        command=self._dp_canvas.yview)

    self._dp_inner = tk.Frame(self._dp_canvas, bg=BG)

    self._dp_inner.bind("<Configure>",

        lambda e: self._dp_canvas.configure(

            scrollregion=self._dp_canvas.bbox("all")))

    self._dp_canvas.create_window((0,0), window=self._dp_inner, anchor="nw")

    self._dp_canvas.configure(xscrollcommand=hscroll.set,

        yscrollcommand=vscroll.set)

    self._dp_canvas.pack(side="left", fill="both", expand=True)

    vscroll.pack(side="right", fill="y")

    hscroll.pack(side="bottom", fill="x")

    self._dp_detail = tk.Label(t, text="", bg=TRACE_BG, fg=TEXT_DARK,

```

```

        font=("Courier", 9), anchor="w", padx=8, pady=4)

    self._dp_detail.pack(fill="x", padx=10, pady=(0,6))

    def _build_trace_tab(self):

        t = self._tab_trace

        ctrl = tk.Frame(t, bg=BG)

        ctrl.pack(fill="x", padx=10, pady=8)

        self._btn(ctrl, "◀ Prev", self._trace_prev, PANEL,
TEXT_DARK).pack(side="left", padx=4)

        self._btn(ctrl, "▶ Next", self._trace_next, ACC_MED,
"#F5F0EB").pack(side="left", padx=4)

        self._btn(ctrl, "▶ Auto Play", self._trace_auto, ACC_DARK,
"#F5F0EB").pack(side="left", padx=4)

        self._btn(ctrl, "■ Stop", self._trace_stop, PANEL, TEXT_DARK).pack(side="left",
padx=4)

        self._trace_progress = tk.Label(ctrl, text="Step 0 / 0", bg=BG, fg=TEXT_MID,

        font=("Courier", 9))

        self._trace_progress.pack(side="right", padx=10)

        tf = tk.Frame(t, bg=BG)

        tf.pack(fill="both", expand=True, padx=10, pady=4)

        self._trace_text = tk.Text(tf, bg=TRACE_BG, fg=TEXT_DARK,

        font=("Courier", 10), relief="flat",

        state="disabled", wrap="word",

        padx=10, pady=8)

        ts = ttk.Scrollbar(tf, orient="vertical", command=self._trace_text.yview)

        self._trace_text.configure(yscrollcommand=ts.set)

```

```

self._trace_text.pack(side="left", fill="both", expand=True)

ts.pack(side="right", fill="y")

self._trace_text.tag_configure("header", font=("Courier", 11, "bold"),
                                foreground=ACC_DARK)

self._trace_text.tag_configure("highlight", background=ACC_MED,
                                foreground="#F5F0EB")

self._trace_text.tag_configure("best", foreground=OBST_NODE,
                                font=("Courier", 10, "bold"))

self._trace_text.tag_configure("normal", foreground=TEXT_DARK)

self._trace_text.tag_configure("dim", foreground=TEXT_LIGHT)

def _get_data(self):

    keys, freqs = [], []

    for ke, fe in zip(self._keys_entries, self._freq_entries):

        k = ke.get().strip()

        f = fe.get().strip()

        if k and f:

            try:

                keys.append(float(k))

                freqs.append(float(f))

            except ValueError:

                return None, None

    return keys, freqs

def _run_obst(self):

    keys, freqs = self._get_data()

```

```

if not keys:

    messagebox.showwarning("No Data", "Please enter keys and frequencies.")

    return

if len(keys) < 2:

    messagebox.showwarning("Too Few", "Enter at least 2 keys.")

    return

n = len(keys)

e, w, root_table = obst_dp(keys, freqs)

obst_tree = build_tree(root_table, 1, n, keys)

bst_tree = build_balanced_bst(keys)

self._last_keys = keys

self._last_freq = freqs

self._last_root = root_table

total_f = sum(freqs)

norm_freq = [f/total_f for f in freqs]

obst_cost = e[1][n] / total_f

bst_cost = avg_search_cost(bst_tree, keys, freqs)

self._obst_cost_lbl.config(

    text=f"Avg Search Cost: {obst_cost:.4f} {'★ Better' if obst_cost<=bst_cost else

"}")

self._bst_cost_lbl.config(

    text=f"Avg Search Cost: {bst_cost:.4f} {'★ Better' if bst_cost<obst_cost else

"}")

```

```

        self._draw_tree(obst_tree, self._canvas_obst, OBST_NODE, OBST_TEXT,
ACC_DARK)

```

```

        self._draw_tree(bst_tree, self._canvas_bst, BST_NODE, BST_TEXT,
ACC_MED)

```

```

        self._build_dp_grid(e, w, root_table, n, keys)

```

```

        self._build_trace(e, w, root_table, n, keys)

```

```

        self._status.config(text=f"Done. n={n} keys. OBST cost={obst_cost:.4f}")

```

```

        self._nb.select(self._tab_tree)

```

```

def _draw_tree(self, tree, canvas, node_color, text_color, edge_color):

```

```

    canvas.delete("all")

```

```

    if tree is None:

```

```

        return

```

```

    counter = [0]

```

```

    assign_positions(tree, 0, counter)

```

```

    n_nodes = counter[0]

```

```

    h = tree_height(tree)

```

```

    canvas.update_idletasks()

```

```

    W = canvas.winfo_width() or 600

```

```

    H = canvas.winfo_height() or 400

```

```

    pad_x = max(30, W // (n_nodes + 1))

```

```

    pad_y = max(40, (H - 30) // (h + 1))

```

```

    r = min(22, pad_x // 2 - 2, pad_y // 2 - 4)

```

```

    def pos(node):

```

```

        x = (node["x_order"] + 1) * (W / (n_nodes + 1))

```

```

        y = (node["depth"] + 1) * pad_y

    return x, y

def draw_node(node):

    if node is None:

        return

    x, y = pos(node)

    if node["left"]:

        cx, cy = pos(node["left"])

        canvas.create_line(x, y, cx, cy, fill=edge_color, width=2)

        draw_node(node["left"])

    if node["right"]:

        cx, cy = pos(node["right"])

        canvas.create_line(x, y, cx, cy, fill=edge_color, width=2)

        draw_node(node["right"])

    canvas.create_oval(x-r, y-r, x+r, y+r, fill=node_color,

                      outline=edge_color, width=2)

    k = node["key"]

    label = str(int(k)) if k == int(k) else str(round(k, 2))

    canvas.create_text(x, y, text=label, fill=text_color,

                      font=("Courier", 9, "bold"))

draw_node(tree)

def _highlight_subtree(self, canvas, tree, i, j, keys, node_color, edge_color):

    self._draw_tree(tree, canvas, node_color, OBST_TEXT, edge_color)

    if tree is None:

```

```

    return

n_nodes = sum(1 for _ in self._iter_nodes(tree))

h = tree_height(tree)

canvas.update_idletasks()

W = canvas.winfo_width() or 600

H = canvas.winfo_height() or 400

pad_y = max(40, (H - 30) // (h + 1))

r = min(22, 28)

target_keys = set(keys[k-1] for k in range(i, j+1))

def recolor(node):

    if node is None:

        return

    recolor(node["left"])

    recolor(node["right"])

    if node["key"] in target_keys:

        x = (node["x_order"] + 1) * (W / (n_nodes + 1))

        y = (node["depth"] + 1) * pad_y

        canvas.create_oval(x-r, y-r, x+r, y+r,

                           fill=ACC_LIGHT, outline=ACC_DARK, width=3)

        k = node["key"]

        label = str(int(k)) if k == int(k) else str(round(k, 2))

        canvas.create_text(x, y, text=label, fill=TEXT_DARK,

                           font=("Courier", 10, "bold"))

recolor(tree)

```

```

def _iter_nodes(self, node):

    if node is None:

        return

    yield node

    yield from self._iter_nodes(node["left"])

    yield from self._iter_nodes(node["right"])

def _build_dp_grid(self, e, w, root_table, n, keys):

    for widget in self._dp_inner.winfo_children():

        widget.destroy()

    self._dp_cells.clear()

    CELL_W = 72

    CELL_H = 38

    max_e = max(e[i][j] for i in range(1, n+1) for j in range(i, n+1))

    min_e = min(e[i][j] for i in range(1, n+1) for j in range(i, n+1))

    def heat_color(val):

        if max_e == min_e:

            return HEAT[0]

        idx = int((val - min_e) / (max_e - min_e) * (len(HEAT)-1))

        return HEAT[max(0, min(idx, len(HEAT)-1))]

    tk.Label(self._dp_inner, text="i \ j", bg=BG, fg=TEXT_MID,

            font=("Courier", 8, "bold"),

            width=6, height=2, relief="flat").grid(row=0, column=0, padx=1, pady=1)

    for j in range(1, n+1):

        kj = keys[j-1]

```

```

kstr = str(int(kj)) if kj == int(kj) else str(round(kj,2))

tk.Label(self._dp_inner, text=f"j={j}\n({kstr})", bg=PANEL, fg=TEXT_MID,
        font=("Courier", 7), width=9, height=2,
        relief="flat").grid(row=0, column=j, padx=1, pady=1)

for i in range(1, n+1):

    ki = keys[i-1]

    kstr = str(int(ki)) if ki == int(ki) else str(round(ki,2))

    tk.Label(self._dp_inner, text=f"i={i}\n({kstr})", bg=PANEL, fg=TEXT_MID,
            font=("Courier", 7), width=6, height=2,
            relief="flat").grid(row=i, column=0, padx=1, pady=1)

    for j in range(1, n+1):

        if j < i:

            lbl = tk.Label(self._dp_inner, text="", bg=BG,
                           width=9, height=2, relief="flat")

            lbl.grid(row=i, column=j, padx=1, pady=1)

        else:

            cost = e[i][j]

            root_k = root_table[i][j]

            rk = keys[root_k-1]

            rkstr = str(int(rk)) if rk == int(rk) else str(round(rk,2))

            bg = heat_color(cost)

            fg = "#F5F0EB" if HEAT.index(bg) > 3 else TEXT_DARK

            text = f"{cost:.2f}\nr={rkstr}"

            lbl = tk.Label(self._dp_inner, text=text,

```

```

        bg=bg, fg=fg,

        font=("Courier", 7), width=9, height=2,

        relief="flat", cursor="hand2")

    lbl.grid(row=i, column=j, padx=1, pady=1)

    self._dp_cells[(i, j)] = lbl

    lbl.bind("<Button-1>",

            lambda e, ii=i, jj=j: self._dp_cell_click(ii, jj))

    lbl.bind("<Enter>",

            lambda e, lbl=lbl, bg=bg: lbl.config(

                relief="sunken", bd=1))

    lbl.bind("<Leave>",

            lambda e, lbl=lbl: lbl.config(relief="flat", bd=0))

def _dp_cell_click(self, i, j):

    if not self._last_keys:

        return

    keys = self._last_keys

    root_table = self._last_root

    e, w, _ = obst_dp(keys, self._last_freq)

    root_k = root_table[i][j]

    rk = keys[root_k-1]

    rkstr = str(int(rk)) if rk == int(rk) else str(round(rk, 2))

    ki_str = str(int(keys[i-1])) if keys[i-1]==int(keys[i-1]) else str(round(keys[i-1],2))

    kj_str = str(int(keys[j-1])) if keys[j-1]==int(keys[j-1]) else str(round(keys[j-1],2))

    self._dp_detail.config(

```

```

text=f" Range [{i},{j}] keys {ki_str}..{kj_str} | "

        f"Optimal Root: {rkstr} | Cost  $e[{i}][{j}] = {e[i][j]:.4f}$  | "

        f"Weight  $w = {w[i][j]:.4f}$ ")

obst_tree = build_tree(root_table, 1, len(keys), keys)

counter = [0]

assign_positions(obst_tree, 0, counter)

self._highlight_subtree(self._canvas_obst, obst_tree, i, j,

                        keys, OBST_NODE, ACC_DARK)

self._nb.select(self._tab_tree)

def _build_trace(self, e, w, root_table, n, keys):

    self._trace_steps = []

    def ks(idx):

        k = keys[idx-1]

        return str(int(k)) if k == int(k) else str(round(k, 2))

    self._trace_steps.append({

        "header": "OBST — Algorithm Overview",

        "body": (

            f"We have  $n={n}$  keys: {'', '.join(ks(i+1) for i in range(n))}\n\n"

            "The standard  $O(n^3)$  dynamic programming algorithm fills a table\n"

            " $e[i][j]$  = minimum expected search cost for keys  $i..j$ .\n\n"

            "We iterate over chain lengths  $L = 1, 2, \dots, n$ .\n"

            "For each range  $[i, j]$ , we try every possible root  $r$  in  $[i, j]$ \n"

            "and pick the one with minimum cost."

```

```

    ),

    "highlight": []

})

for length in range(1, n+1):

    self._trace_steps.append({

        "header": f"Chain Length L = {length}",

        "body": (

            f"Solving all sub-problems of length {length}.\n"

            f"There are {n-length+1} sub-problem(s) of this length.\n"

            f"Each sub-problem tests up to {length} root candidate(s)."

        ),

        "highlight": []

    })

    for i in range(1, n-length+2):

        j = i + length - 1

        best_r = root_table[i][j]

        lines = [f"Sub-problem [{i}, {j}] keys {ks(i)}..{ks(j)}\n",

            f"Weight  $w[{i}][{j}] = \{w[i][j]:.4f\}\n\n",

            "Testing each root r:\n"]

        candidates = []

        for r in range(i, j+1):

            cost_r = e[i][r-1] + e[r+1][j] + w[i][j]

            marker = " ◀ BEST" if r == best_r else ""

            candidates.append((r, cost_r, marker))$ 
```

```

        lines.append(f" r = {ks(r)} : e[{i}][{r-1}] + e[{r+1}][{j}] + {w[i][j]:.4f}")

        f' = {cost_r:.4f} {marker}\n")

    lines.append(f"\n→ Optimal root for [{i},{j}] = {ks(best_r)}")

    lines.append(f"\n→ e[{i}][{j}] = {e[i][j]:.4f}")

    self._trace_steps.append({

        "header": f"L={length} · Range [{i}, {j}]",

        "body": "".join(lines),

        "highlight": [(i, j)],

        "best_r": best_r,

        "candidates": candidates

    })

    self._trace_steps.append({

        "header": "Result",

        "body": (

            f"Final answer: e[1][{n}] = {e[1][n]:.4f}\n\n"

            f"Total sum of frequencies = {sum(self._last_freq):.4f}\n"

            f"Average search cost = {e[1][n]/sum(self._last_freq):.4f}\n\n"

            "The OBST is stored in root[i][j] table.\n"

            "Navigate to the 'Tree Comparison' tab to see both trees."

        ),

        "highlight": []

    })

    self._trace_idx = 0

    self._render_trace_step()

```

```

def _render_trace_step(self):

    if not self._trace_steps:

        return

    step = self._trace_steps[self._trace_idx]

    total = len(self._trace_steps)

    self._trace_progress.config(

        text=f"Step {self._trace_idx+1} / {total}")

    t = self._trace_text

    t.configure(state="normal")

    t.delete("1.0", "end")

    t.insert("end", step["header"] + "\n", "header")

    t.insert("end", "—" * 60 + "\n\n", "dim")

    body = step.get("body", "")

    for line in body.splitlines(keepends=True):

        if "◀ BEST" in line:

            t.insert("end", line, "best")

        else:

            t.insert("end", line, "normal")

    t.configure(state="disabled")

    for cell_lbl in self._dp_cells.values():

        bg = cell_lbl.cget("bg")

        cell_lbl.config(relief="flat", bd=0)

    for (ci, cj) in step.get("highlight", []):

        if (ci, cj) in self._dp_cells:

```

```

        self._dp_cells[(ci, cj)].config(relief="ridge", bd=3)

def _trace_prev(self):
    if self._trace_idx > 0:
        self._trace_idx -= 1
        self._render_trace_step()

def _trace_next(self):
    if self._trace_idx < len(self._trace_steps) - 1:
        self._trace_idx += 1
        self._render_trace_step()

def _trace_auto(self):
    self._trace_running = True
    self._auto_step()

def _auto_step(self):
    if not self._trace_running:
        return

    if self._trace_idx < len(self._trace_steps) - 1:
        self._trace_idx += 1
        self._render_trace_step()
        self.after(900, self._auto_step)
    else:
        self._trace_running = False

def _trace_stop(self):
    self._trace_running = False

def _reset_all(self):

```

```

for frame in self._row_frames:

    frame.destroy()

self._keys_entries.clear()

self._freq_entries.clear()

self._row_frames.clear()

self._canvas_obst.delete("all")

self._canvas_bst.delete("all")

for w in self._dp_inner.winfo_children():

    w.destroy()

self._dp_cells.clear()

self._trace_steps.clear()

self._trace_idx = 0

self._last_keys.clear()

self._last_freq.clear()

self._last_root = None

t = self._trace_text

t.configure(state="normal")

t.delete("1.0", "end")

t.configure(state="disabled")

self._status.config(text="Reset.")

self._obst_cost_lbl.config(text="Avg Search Cost: —")

self._bst_cost_lbl.config(text="Avg Search Cost: —")

self._dp_detail.config(text="")

self._add_row()

```

```

        self._add_row()

        self._add_row()

if __name__ == "__main__":

    app = OBSTApp()

    example = [("10","3"),("20","3"),("30","1"),("40","1")]

    for frame in list(app._row_frames):

        frame.destroy()

    app._keys_entries.clear()

    app._freq_entries.clear()

    app._row_frames.clear()

    for k, f in example:

        app._add_row(k, f)

    app.mainloop()

```